



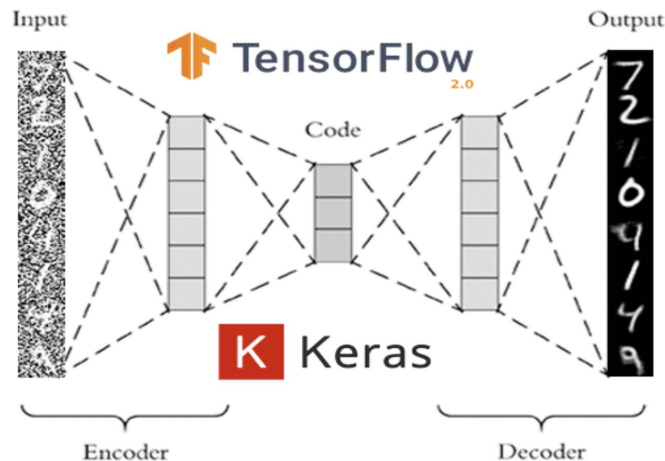
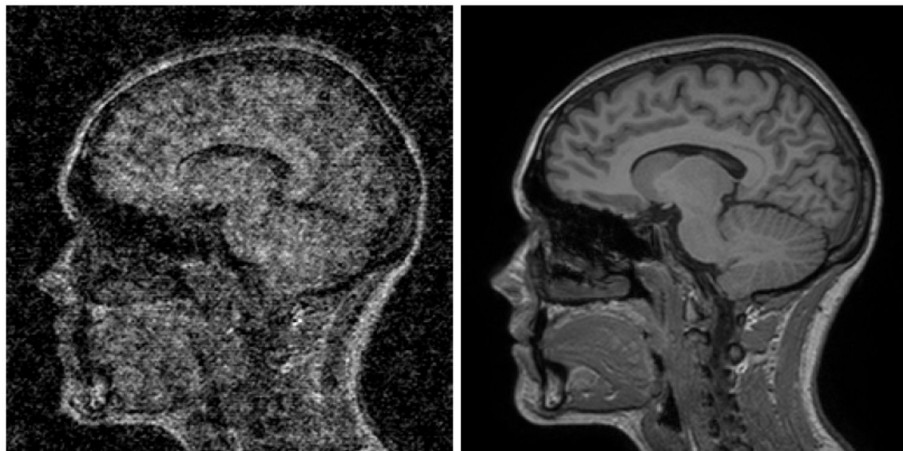
Image Denoising with Autoencoders: A Comparison of CNN, MLP, and Benchmark Architectures

CSCI 611

Cameron Nolen, Chance Stahura, Hugo Tanaka

What Is Image Denoising

- Images are almost never perfectly clean, and have various sources of noise
- Denoising is the task of recovering the clean underlying signal from a corrupted version of an image



Dataset/Preprocessing

- Data:
 - Two standard benchmarks: **BSD500** (Berkeley Segmentation Dataset) for training and validation, and **CBSD68** as the held-out test set
- Preprocessing:
 - Normalization
 - Patch-based training (64x64)
 - Noise injection



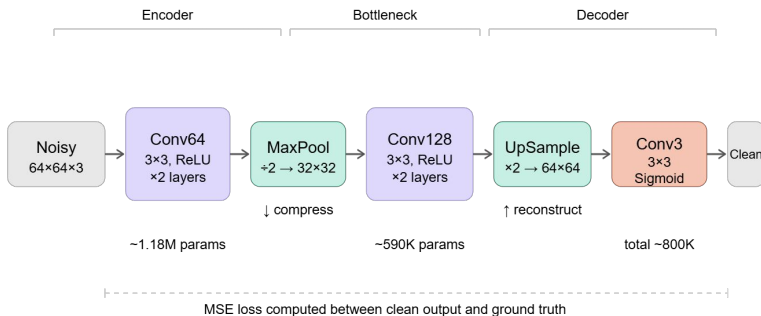
Adapted Models

- Dense (fully connected)
 - No spatial awareness, just dense layers
 - Encoder: $(12288 \times 128) + (128 \times 64) + (64 \times 32) = \sim 1.58\text{M}$ parameters
 - Decoder: roughly symmetric, $\sim 1.58\text{M}$
- Tensorflow benchmark (adapted from TF tutorial)
 - Shallow conv architecture: $16 \rightarrow 8$ filters, $\sim 7,000$ trainable parameters
 - Uses Conv2DTranspose, which learns the upsampling itself as a trainable operation

```
162  def build_dense_model(  
163      input_shape: tuple[  
164          int | None,  
165          int | None,  
166          int,  
167      ] = TRAIN_INPUT_SHAPE,  
168  ) -> tf.keras.Model:  
169      """This will build the fully connected model."""  
170      # (64 x 64) x 3 = 12288  
171      inputs = layers.Input(shape=input_shape)  
172  
173      x = layers.Flatten()(inputs)  
174  
175      # encoder  
176      encoded = layers.Dense(128, activation="relu")(x)  
177      encoded = layers.Dense(64, activation="relu")(encoded)  
178      encoded = layers.Dense(32, activation="relu")(encoded)  
179  
180      # decoder  
181      decoded = layers.Dense(64, activation="relu")(encoded)  
182      decoded = layers.Dense(128, activation="relu")(decoded)  
183      decoded = layers.Dense(12288, activation="sigmoid")(decoded)  
184  
185      output = layers.Reshape(input_shape)(decoded)  
186  
187      return models.Model(inputs, output, name="dense_autoencoder")
```

Our Model

- Encoder: two conv blocks → MaxPooling
- Bottleneck
- Decoder: UpSampling → final conv
- 700K–800K trainable parameters



```
"""This will build the auto_encoder model."""
inputs = layers.Input(shape=input_shape)

x: tf.Tensor = layers.Conv2D(
    FILTERS_STAGE_1, CONV_KERNEL_SIZE, activation="relu", padding="same"
)(inputs)
x = layers.Conv2D(
    FILTERS_STAGE_1, CONV_KERNEL_SIZE, activation="relu", padding="same"
)(x)
x = layers.MaxPooling2D(POOL_FACTOR, padding="same")(x)

x = layers.Conv2D(
    FILTERS_STAGE_2, CONV_KERNEL_SIZE, activation="relu", padding="same"
)(x)
x = layers.Conv2D(
    FILTERS_STAGE_2, CONV_KERNEL_SIZE, activation="relu", padding="same"
)(x)
x = layers.UpSampling2D(UPSAMPLE_FACTOR)(x)

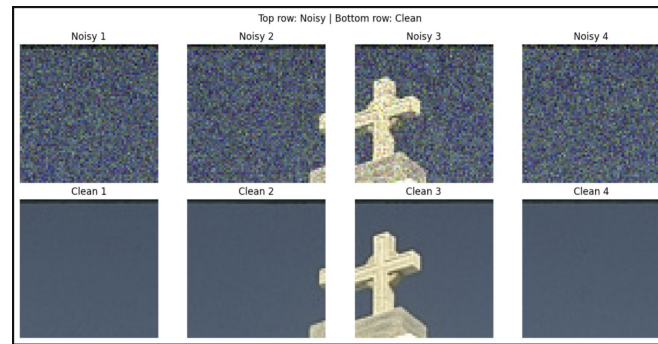
outputs = layers.Conv2D(
    INPUT_CHANNELS, CONV_KERNEL_SIZE, activation="sigmoid", padding="same"
)(x)

return models.Model(inputs, outputs, name="denoising_autoencoder")
```

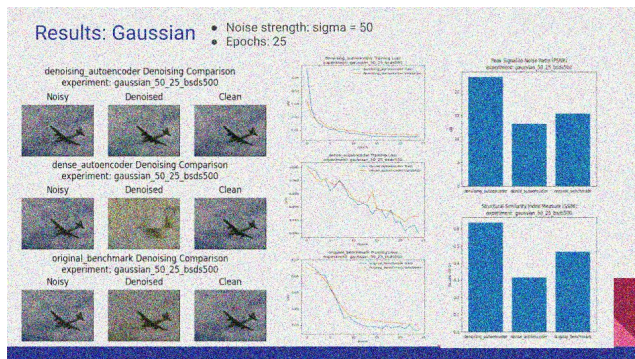
Training Setup

	Dense	TF benchmark	CNN
Optimizer	Adam 1e-3	Adam 1e-3	Adam 1e-3
Loss	MSE	MSE	MSE
Epochs	25	25	25
Batch size	32	32	32
Test Mode	Patch (64x64)	Patch (64x64)	Full image

```
Noisy shape: (35, 64, 64, 3)
Clean shape: (35, 64, 64, 3)
Total patches from image: 4096
```



Example

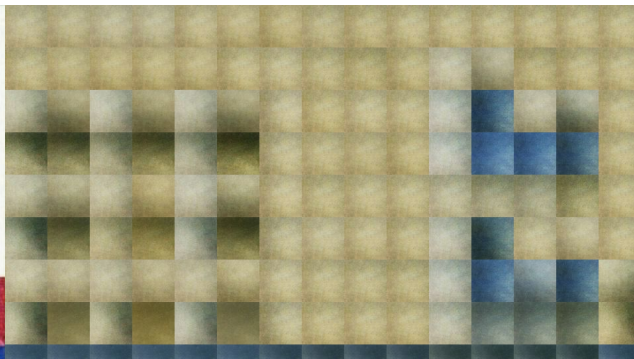
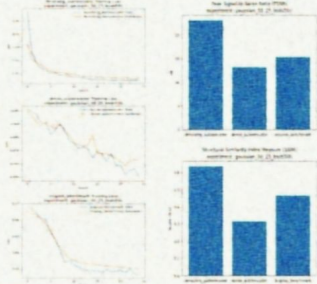
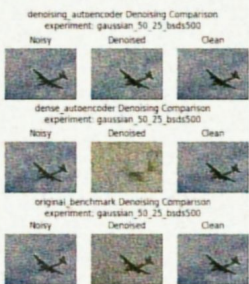


CNN

MLP

TF Benchmark

Results: Gaussian • Noise strength: sigma = 50
• Epochs: 25



Results: Gaussian

- Noise strength: sigma = 50
- Epochs: 25

denoising_autoencoder Denoising Comparison
experiment: gaussian_50_25_bsds500

Noisy

Denoised

Clean



dense_autoencoder Denoising Comparison
experiment: gaussian_50_25_bsds500

Noisy

Denoised

Clean

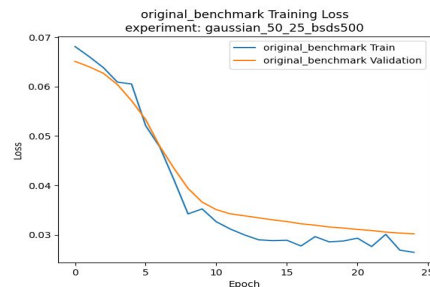
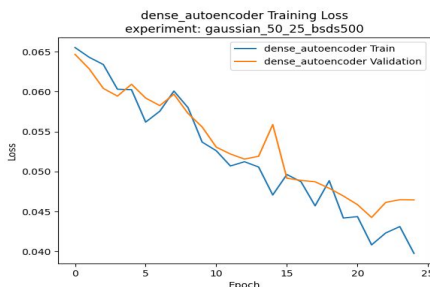
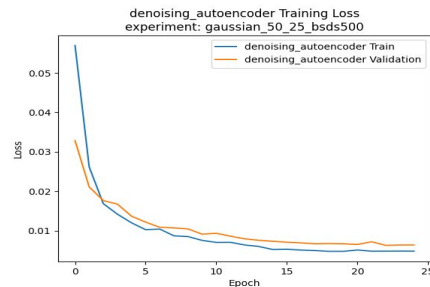


original_benchmark Denoising Comparison
experiment: gaussian_50_25_bsds500

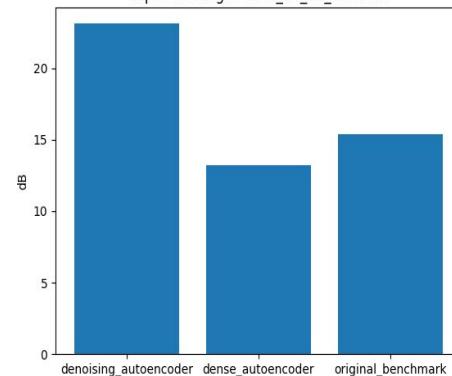
Noisy

Denoised

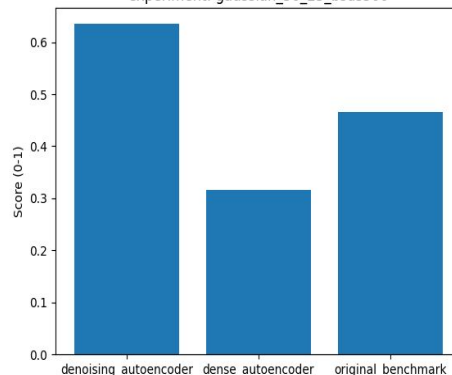
Clean



Peak Signal-to-Noise Ratio (PSNR)
experiment: gaussian_50_25_bsds500



Structural Similarity Index Measure (SSIM)
experiment: gaussian_50_25_bsds500



Results: Salt and Pepper

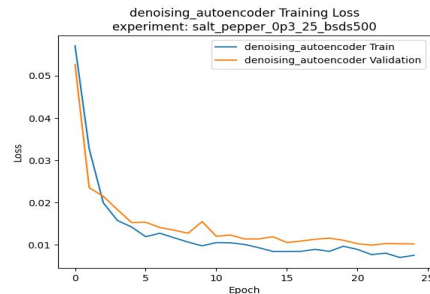
- Noise strength: $p = 0.3$
- Epochs: 25

denoising_autoencoder Denoising Comparison
experiment: salt_pepper_0p3_25_bsds500

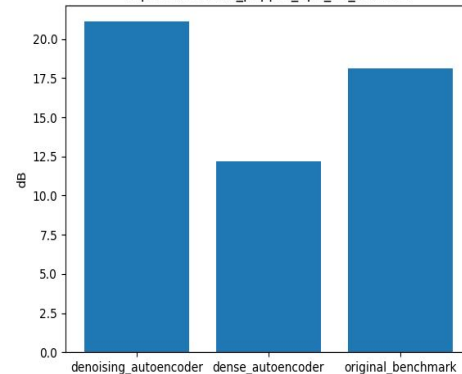
Noisy

Denoised

Clean



Peak Signal-to-Noise Ratio (PSNR)
experiment: salt_pepper_0p3_25_bsds500



dense_autoencoder Denoising Comparison
experiment: salt_pepper_0p3_25_bsds500

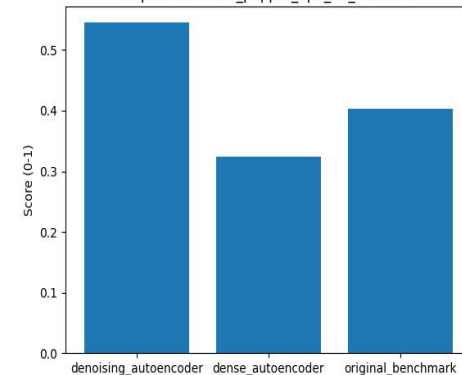
Noisy

Denoised

Clean



Structural Similarity Index Measure (SSIM)
experiment: salt_pepper_0p3_25_bsds500

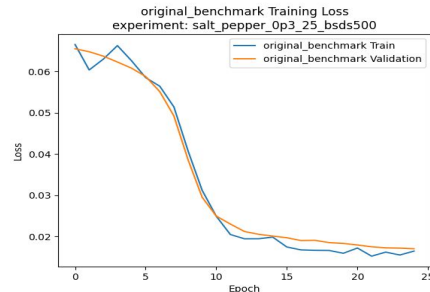


original_benchmark Denoising Comparison
experiment: salt_pepper_0p3_25_bsds500

Noisy

Denoised

Clean



Results: Occlusion

- Noise strength: size = 32x32
- Epochs: 25

denoising_autoencoder Denoising Comparison
experiment: occlusion_32_25_bsds500

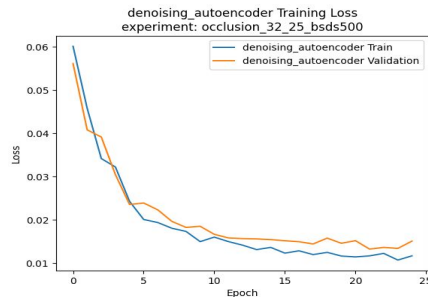
Noisy



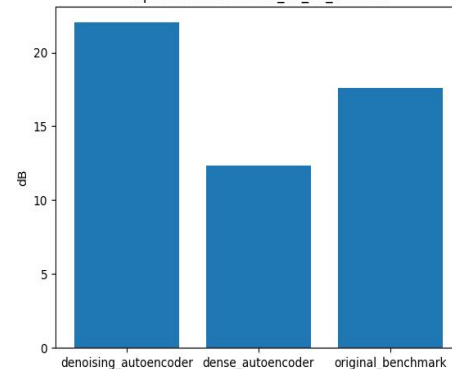
Denoised



Clean



Peak Signal-to-Noise Ratio (PSNR)
experiment: occlusion_32_25_bsds500

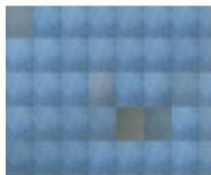


dense_autoencoder Denoising Comparison
experiment: occlusion_32_25_bsds500

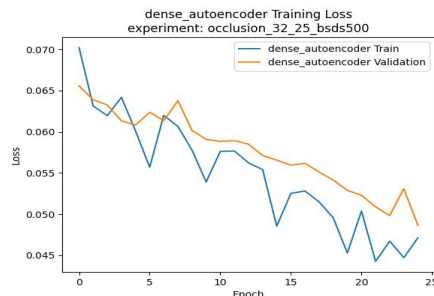
Noisy



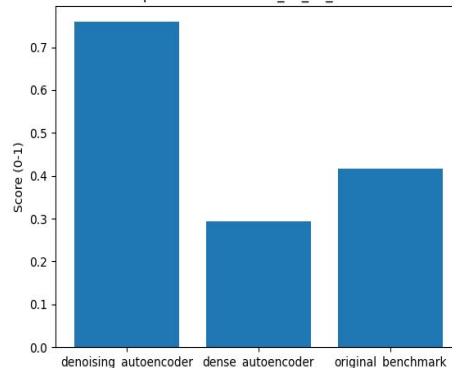
Denoised



Clean



Structural Similarity Index Measure (SSIM)
experiment: occlusion_32_25_bsds500



original_benchmark Denoising Comparison
experiment: occlusion_32_25_bsds500

Noisy



Denoised



Clean



Analysis

- Our model had the most accurate and consistency out of the other two test models.
- Each model had a distinct “personalities”:
 - Our model: highly recoverable, but not so well when the noise becomes extremely outstanding in the image.
 - Dense model: Instead of learning to denoise, it **regressed to the mean**. It predicts a safe, but blurry average rather than recovering the actual image.
 - Original TensorFlow’s CNN model: image does appear, but misses crucial color details, and the noise handling is not very good.

